- 📘 DOCUMENTACIÓN TÉCNICA — GALA6 Chocoletra Bot
  - Módulo 1: Estrategia Central (core/strategy.py)
  - 🎯 ¿Qué hace este módulo?
  - 📐 Flujo de Cálculo (10 Pasos)
  - 🔧 Parámetros Configurables (config.yaml)
  - 📊 Columnas del DataFrame de Salida
  - 💡 Equivalencia ThinkScript → Python
  - 🧪 Cómo Probar Este Módulo

📘 DOCUMENTACIÓN TÉCNICA — GALA6 Chocoletra Bot

Módulo 1: Estrategia Central (core/strategy.py)

🎯 ¿Qué hace este módulo?

Este es el **corazón del sistema**. Traduce exactamente la lógica del script `GALA6 LINEAL CORREGIDO` de ThinkScript a Python puro.

Recibe velas de mercado (OHLCV) y retorna señales de trading: `BUY`, `SELL`, `OM-R` (omitida por rango), `OM-T` (omitida por tendencia).

📐 Flujo de Cálculo (10 Pasos)

```
Velas OHLCV
    │
    ▼
[1] MACD ──────────────────────── EMA12 - EMA26 → Señal(9) → Histograma
    │
    ▼
[2] ATR Wilder-14 ──────────────── True Range → Media Exponencial (α=1/14)
    │
```

```
    ▼
[3] Tendencia EMA-11 ──────────── Pendiente mínima = ATR × 0.02
    │
    ▼
[4] Memoria del Cruce ──────────── crossHigh / crossLow / crossBars / dirección
    │
    ▼
[5] Filtro de Rango ──────────── 8 velas atrás → Rango % ≥ 0.18%
    │
    ▼
[6] Histograma ──────────── Progresivo (3 velas) + Fuerte (> umbral ATR)
    │
    ▼
[7] Impulso de Vela ──────────── Cuerpo > 60% ATR + Cierre dominante
    │
    ▼
[8] Breakout ──────────── Precio > crossHigh + buffer ATR
    │
    ▼
[9] Señales Base ──────────── Normal (≥2 velas) o Rápida (≥1 vela)
    │
    ▼
[10] Cooldown 8 Velas ──────────── Bloqueo de señales repetidas
    │
    ▼
Señal Final: BUY / SELL / OM-R / OM-T / None
```

🔧 Parámetros Configurables (config.yaml)

Parámetro	Valor Default	Descripción
`fast_length`	12	Período EMA rápida del MACD
`slow_length`	26	Período EMA lenta del MACD
`signal_length`	9	Período de la línea de señal MACD
`ema_trend_length`	11	EMA de tendencia
`ema_slope_factor`	0.02	Factor mínimo de pendiente (× ATR)
`range_lookback`	8	Velas hacia atrás para el filtro de rango
`min_range_pct`	0.18	Rango mínimo válido (%)

Parámetro	Valor Default	Descripción
`breakout_buffer_factor`	0.02	Buffer de ruptura (× ATR)
`hist_threshold_factor`	0.04	Umbral del histograma (× ATR)
`impulse_body_factor`	0.60	Factor del cuerpo de impulso (× ATR)
`allow_fast_entry`	true	Permitir entrada rápida
`cooldown_bars`	8	Velas de bloqueo tras señal

📊 Columnas del DataFrame de Salida

Columna	Tipo	Descripción
`macd_line`	float	Línea MACD (EMA12 - EMA26)
`signal_line`	float	Línea de señal (EMA del MACD)
`hist_line`	float	Histograma (macd - signal)
`atr`	float	ATR Wilder-14
`ema_trend`	float	EMA de tendencia (período 11)
`ema_slope`	float	Pendiente de la EMA de tendencia
`valid_range`	bool	True si el rango de precio es suficiente
`cross_high`	float	Máximo en el último cruce alcista
`cross_low`	float	Mínimo en el último cruce bajista
`cross_bars`	int	Velas desde el último cruce
`direccion_activa`	int	1=alcista, -1=bajista
`bull_hist_progressive`	bool	Histograma progresivo alcista
`bear_hist_progressive`	bool	Histograma progresivo bajista
`fast_bull_impulse`	bool	Vela de impulso alcista rápido
`fast_bear_impulse`	bool	Vela de impulso bajista rápido
`bull_break`	bool	Ruptura del muro alcista

Columna	Tipo	Descripción
`bear_break`	bool	Ruptura del muro bajista
`buy_signal_raw`	bool	Señal BUY sin cooldown
`sell_signal_raw`	bool	Señal SELL sin cooldown
`omitida_buy_rango`	bool	Señal BUY omitida por rango
`omitida_sell_rango`	bool	Señal SELL omitida por rango
`omitida_buy_tendencia`	bool	Señal BUY omitida por tendencia
`omitida_sell_tendencia`	bool	Señal SELL omitida por tendencia
`señal`	str/None	Señal final: BUY/SELL/OM-R/OM-T/None

💡 Equivalencia ThinkScript → Python

ThinkScript	Python (strategy.py)
`ExpAverage(close, 12)`	`df['close'].ewm(span=12, adjust=False).mean()`
`TrueRange(high, close, low)`	`pd.concat([tr1, tr2, tr3], axis=1).max(axis=1)`
`MovingAverage(AverageType.WILDERS, tr, 14)`	`true_range.ewm(alpha=1/14, adjust=False).mean()`
`Highest(high[1], 8)`	`df['high'].shift(1).rolling(8).max()`
`CompoundValue(1, if bull then 0 else x[1]+1, 0)`	Bucle `for i in range(n)` con estado acumulado
`macdLine crosses above signalLine`	`(macd > signal) & (macd.shift(1) <= signal.shift(1))`

🧪 Cómo Probar Este Módulo

```python
import pandas as pd
import yaml
from core.strategy import GALA6Strategy

# Cargar configuración
with open("config.yaml") as f:
    config = yaml.safe_load(f)

# Crear estrategia
strategy = GALA6Strategy(config)

# Cargar datos (CSV con columnas: open, high, low, close, volume)
df = pd.read_csv("data/BA_1min.csv", index_col="datetime", parse_dates=True)

# Calcular señales
resultado = strategy.calcular(df)

# Ver señales generadas
señales = resultado[resultado['señal'].notna()]
print(señales[['close', 'atr', 'hist_line', 'señal']])
```
